λ_S^1 — Method Return Values:

Extending λ_S^0 with Non-Session-Typed Returns

Alexandre Zua Caldeira
Independent Researcher
azuacaldeira@gmail.com

March 2026

Abstract

λ_S^0 [1] formalises straight-line typestate checking where every method returns **Unit**. Real Java methods return values: `checkBalance()` returns `double`, `read()` returns `String`. We present λ_S^1, a conservative extension of λ_S^0 with base-type return values (**Int**, **Double**, **String**, **Bool**) and a method-signature table *msig*. We prove that return types are *orthogonal* to the session discipline: every λ_S^0 derivation is valid in λ_S^1, and the soundness theorems (preservation, progress, session completion) carry over with minimal modification. The key insight—formalised as the *Orthogonality Theorem*—is that base-type bindings never enter the session-typed environment Γ, so the transition function δ and the "all at $\perp$" check are entirely unaffected. This corresponds precisely to the fact that BICA Reborn's `TypestateChecker` already ignores Java return types.

1 Introduction

In the λ_S calculus hierarchy, λ_S^0 [1] captures the simplest possible typestate scenario: straight-line code on session-typed objects, where every method call returns **Unit** (the value **skip**). This is a useful starting point for establishing the formal correspondence with the BICA Reborn tool [2], but it is clearly unrealistic: Java methods routinely return values of base types.

The question λ_S^1 answers is: *does allowing method calls to return base-type values disturb the session discipline?* The answer is no. Informally, the session-typed environment Γ tracks only bindings of the form $x : C \mathbin{@} s$ (object x of class C at protocol state s). When a method call $x.m()$ returns a value $c : \tau$ for some base type τ, the variable that receives c (via **let**) enters the typing context with type τ—but this is a *non-session* binding that has no protocol state and participates in none of the session-related checks.

Concretely, λ_S^1 extends λ_S^0 with:

1. **Base types** $\tau ::=$ **Int** | **Double** | **String** | **Bool**.
2. **A method-signature table** $msig : CName \times MName \to \tau$, recording the return type of each method.
3. **Constants** c as values, with a typing rule T-Const.
4. **A modified T-Call** that returns $\tau = msig(C, m)$ instead of **Unit**.

The session type grammar, state-space construction, lattice structure, and transition function δ are *completely unchanged*.

Correspondence with VGR 2006. The orthogonality of return types and session state was first observed in the channel setting by Vasconcelos, Gay, and Ravara [4], whose typing judgement separates the result type T from the channel environment Σ. λ_S^1 establishes the analogous result for the object setting: the return type lives in the type T of the judgement, while the session state change lives in the environment $\Gamma \to \Gamma'$.

Structure. Section 2 recalls λ_S^0. Section 3 introduces the new syntax. Section 4 presents the type system (six rules). Section 5 gives the operational semantics. Section 6 proves soundness as a conservative extension. Section 7 works through FileHandle and ATM examples. Section 8 discusses the result and previews λ_S^2.

2 λ_S^0 Recap

We briefly recall the λ_S^0 calculus from [1]. A *class table* $\mathcal{CT}$ maps each class name C to a state space $\mathcal{L}(S_C)$ built from the session type S_C. Session types use only Branch and End:

$$S \ ::= \ \&\{m_1 : S_1, \ \ldots, \ m_n : S_n\} \ \mid \ \textbf{end}$$

The state space has a top state $\top$ (initial), bottom state $\bot$ (terminated), and a partial transition function $\delta(s, m) = s'$.

The typing judgement has the form $\mathcal{CT};\ \Gamma \vdash e : T \dashv \Gamma'$ with five rules: T-Skip, T-Var, T-New, T-Call, and T-Let. The key rule is:

$$\text{T-Call}^0 \ \frac{\Gamma(x) = C \mathbin{@} s \qquad \delta(s, m) = s'}{\mathcal{CT};\ \Gamma \vdash x.m() : \textbf{Unit} \dashv \Gamma[x \mapsto C \mathbin{@} s']}$$

Every method call returns **Unit**. The operational semantics has four rules (R-New, R-Call, R-Let-Ctx, R-Let-Val), and soundness is established via preservation, progress, and session completion.

3 λ_S^1 Syntax

3.1 Base Types

We introduce a set of base types:

$$\tau \ ::= \ \textbf{Int} \ \mid \ \textbf{Double} \ \mid \ \textbf{String} \ \mid \ \textbf{Bool}$$

These are the types of scalar values that methods may return. We emphasise that base types carry no session information: there is no "$\tau \mathbin{@} s$" construct.

3.2 Extended Type Grammar

The type grammar extends from λ_S^0 with base types:

$$T \ ::= \ \textbf{Unit} \ \mid \ C \ \mid \ \tau$$

Here C is a class type (session-typed) and τ is a base type (non-session-typed).

3.3 Method-Signature Table

Definition 3.1 (Method-Signature Table)**.** The *method-signature table* is a partial function

$$msig : CName \times MName \rightharpoonup \tau$$

When $msig(C, m) = \tau$, method m of class C returns a value of base type τ. When $msig(C, m)$ is undefined, the method returns **Unit** (backward compatible with λ_S^0).

Remark 3.2. The signature table is separate from both the class table $\mathcal{CT}$ (which maps C to a state space) and the transition function δ (which maps (s, m) to s'). This separation reflects the orthogonality principle: $\mathcal{CT}$ and δ concern protocol structure, while *msig* concerns data flow.

3.4 Constants and Values

The value grammar extends with constants:

$$v \ ::= \ x \ | \ \textbf{skip} \ | \ c$$

where c ranges over base-type constants (integer literals, floating-point literals, string literals, boolean literals). A typing function $typeof(c) = \tau$ assigns the unique base type to each constant.

3.5 Expressions

The expression grammar is unchanged from λ_S^0:

$$e \ ::= \ v \ | \ \textbf{new}\ C() \ | \ x.m() \ | \ \textbf{let}\ x = e_1\ \textbf{in}\ e_2$$

There are no new expression forms. The only change is that $x.m()$ may now evaluate to a constant c instead of **skip**.

Remark 3.3. λ_S^1 deliberately excludes conditionals: there is no **if**–**then**–**else**. This means the client can *bind* a return value but cannot *inspect* it. Conditionals (needed to branch on enum return values for selections) belong to λ_S^2.

4 Type System

The typing judgement retains the same form as λ_S^0:

$$\mathcal{CT};\ \Gamma \vdash e : T \dashv \Gamma'$$

The environment Γ maps session-typed variables to entries $x : C \ @\ s$. Base-type variables are *not* tracked in Γ—they have no protocol state to track. This is the formal expression of the orthogonality principle.

4.1 Typing Rules

We present all six rules. Rules marked with $(\star)$ are new or modified relative to λ_S^0.

$$\textsc{T-Skip}\ \frac{}{\mathcal{CT};\ \Gamma \vdash \textbf{skip} : \textbf{Unit} \dashv \Gamma} \quad \textsc{T-Const}\ (\star)\ \frac{typeof(c) = \tau}{\mathcal{CT};\ \Gamma \vdash c : \tau \dashv \Gamma}$$

T-Const is new. A constant has its base type, and the session-typed environment is unchanged (constants have no protocol relevance).

$$\text{T-Var}\ \frac{\Gamma(x) = C \mathbin{@} s}{\mathcal{CT};\ \Gamma \vdash x : C \dashv \Gamma} \qquad \text{T-New}\ \frac{\mathcal{CT}(C) = \mathcal{L}(S_C) \qquad \top = \mathrm{top}(\mathcal{L}(S_C))}{\mathcal{CT};\ \Gamma \vdash \textbf{new}\ C() : C \dashv \Gamma, x : C \mathbin{@} \top}$$

T-Var and T-New are identical to λ_S^0.

$$\text{T-Call}\ (\star)\ \frac{\Gamma(x) = C \mathbin{@} s \qquad \delta(s, m) = s' \qquad msig(C, m) = \tau}{\mathcal{CT};\ \Gamma \vdash x.m() : \tau \dashv \Gamma[x \mapsto C \mathbin{@} s']}$$

T-Call is modified: the return type is $\tau = msig(C, m)$ instead of **Unit**. When $msig(C, m)$ is undefined, the return type defaults to **Unit** (backward compatibility; see Theorem 6.1). The session-state update $\Gamma[x \mapsto C \mathbin{@} s']$ is *identical* to λ_S^0—the return type does not participate.

$$\text{T-Let}\ \frac{\mathcal{CT};\ \Gamma \vdash e_1 : T_1 \dashv \Gamma_1 \qquad \mathcal{CT};\ \Gamma_1 \vdash e_2 : T_2 \dashv \Gamma_2}{\mathcal{CT};\ \Gamma \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 : T_2 \dashv \Gamma_2}$$

T-Let is unchanged. The key observation: when $e_1 = y.m()$ and $T_1 = \tau$, the variable x is bound to a base-type value. Since x has type τ (not a class type), it does *not* appear in Γ_1. The environment threading is purely through the session-typed part.

4.2 The Orthogonality Principle

Theorem 4.1 (Orthogonality of Return Types)**.** *For any derivation $\mathcal{CT};\ \Gamma \vdash e : T \dashv \Gamma'$ in λ_S^1:*

(i) The output environment Γ' depends only on the session types in $\mathcal{CT}$, the input environment Γ, and the method names in e—never on the method-signature table $msig$ or the return types τ.

(ii) The session-completion predicate $\forall (x : C \mathbin{@} s) \in \Gamma'.\ s = \bot$ is independent of $msig$.

Proof. By structural induction on the derivation. In every rule, the update $\Gamma \mapsto \Gamma'$ depends only on the session-typed part:

- T-Skip, T-Const, T-Var: $\Gamma' = \Gamma$, no dependence on $msig$.
- T-New: $\Gamma' = \Gamma, x : C \mathbin{@} \top$. Depends on $\mathcal{CT}(C)$, not $msig$.
- T-Call: $\Gamma' = \Gamma[x \mapsto C \mathbin{@} s']$ where $\delta(s, m) = s'$. The state s' depends on δ (hence on $\mathcal{CT}$), not on $msig(C, m)$.
- T-Let: $\Gamma' = \Gamma_2$ from the sub-derivations. By induction, each sub-derivation's output environment is independent of $msig$. □

Corollary 4.2. *A typestate checker that tracks only session-typed variables and their states (ignoring method return types entirely) is sound with respect to λ_S^1.*

This is exactly what BICA Reborn's `TypestateChecker` does: it tracks `currentStates` : `Map⟨String, Integer⟩` and never inspects Java return types.

5 Operational Semantics

Configurations are unchanged: $\langle H,\ \sigma,\ e\rangle$ where H is a heap (location $\mapsto$ class), σ is a session store (location $\mapsto$ state), and e is a runtime expression.

Runtime values extend with constants: $v ::= x \mid \textbf{skip} \mid \ell \mid c$.

5.1 Reduction Rules

Three rules are unchanged from λ_S^0: R-New, R-Let-Ctx, and R-Let-Val. Only R-Call is modified.

$$\text{R-New}\ \frac{\mathcal{CT}(C) = \mathcal{L}(S_C) \qquad \ell \notin \mathrm{dom}(H)}{\langle H,\ \sigma,\ \textbf{new}\ C()\rangle \longrightarrow \langle H[\ell \mapsto C],\ \sigma[\ell \mapsto \top],\ \ell\rangle}$$

$$\text{R-Call}\ (\star)\ \frac{H(\ell) = C \qquad \sigma(\ell) = s \qquad \delta(s, m) = s' \qquad \mathit{msig}(C, m) = \tau \qquad c : \tau}{\langle H,\ \sigma,\ \ell.m()\rangle \longrightarrow \langle H,\ \sigma[\ell \mapsto s'],\ c\rangle}$$

In λ_S^0, R-Call produced **skip**. In λ_S^1, it produces a constant c of type $\mathit{msig}(C, m) = \tau$. The constant is chosen nondeterministically—the calculus does not model method bodies, only the protocol discipline. When $\mathit{msig}(C, m)$ is undefined, the rule produces **skip** (backward compatible).

$$\text{R-Let-Ctx}\ \frac{\langle H,\ \sigma,\ e_1\rangle \longrightarrow \langle H',\ \sigma',\ e_1'\rangle}{\langle H,\ \sigma,\ \textbf{let}\ x = e_1\ \textbf{in}\ e_2\rangle \longrightarrow \langle H',\ \sigma',\ \textbf{let}\ x = e_1'\ \textbf{in}\ e_2\rangle}$$

$$\text{R-Let-Val}\ \frac{}{\langle H,\ \sigma,\ \textbf{let}\ x = v\ \textbf{in}\ e_2\rangle \longrightarrow \langle H,\ \sigma,\ e_2[v/x]\rangle}$$

Remark 5.1. The nondeterministic choice of c in R-Call is deliberate: λ_S^1 abstracts over method implementations. For soundness, the specific value does not matter because base-type bindings do not affect the session-typed environment (by Theorem 4.1).

6 Soundness

We establish that λ_S^1 is a conservative extension of λ_S^0 and that the three soundness properties carry over.

6.1 Conservative Extension

Theorem 6.1 (Conservative Extension)**.** *If msig is everywhere undefined (i.e., $\mathit{msig}(C, m) = \bot$ for all C, m), then λ_S^1 reduces to λ_S^0: every λ_S^0 derivation is a valid λ_S^1 derivation, and the reduction semantics coincide.*

Proof. When *msig* is undefined, T-Call returns **Unit** (the default), matching T-Call^0. R-Call produces **skip** (the default), matching the λ_S^0 rule. T-Const is never triggered (no constants in λ_S^0 programs). All other rules are identical. □

6.2 Preservation

Theorem 6.2 (Preservation)**.** *If $\mathcal{CT} \vdash \langle H,\ \sigma,\ e\rangle : T \dashv \Gamma'$ and $\langle H,\ \sigma,\ e\rangle \longrightarrow \langle H_1,\ \sigma_1,\ e_1\rangle$, then there exist T' and Γ_1' such that $\mathcal{CT} \vdash \langle H_1,\ \sigma_1,\ e_1\rangle : T' \dashv \Gamma_1'$.*

Proof. By case analysis on the reduction rule, following the same structure as the λ_S^0 proof:

- R-New: identical to λ_S^0.
- R-Call: the only change is that the result expression is c (of type τ) instead of **skip** (of type **Unit**). The runtime typing environment update is $\Gamma[\ell \mapsto C \mathbin{@} s']$, unchanged from λ_S^0. The result value c has type τ by T-Const, which does not modify Γ.

- R-Let-Ctx: by induction hypothesis on the sub-reduction.
- R-Let-Val: by the substitution lemma. The only new case is substituting a constant c for x: since c has base type τ and is not in Γ, the substitution does not affect the session-typed environment. □

6.3 Progress

Theorem 6.3 (Progress)**.** *If $\mathcal{CT} \vdash \langle H,\ \sigma,\ e\rangle : T \dashv \Gamma'$, then either e is a value or there exists $\langle H',\ \sigma',\ e'\rangle$ such that $\langle H,\ \sigma,\ e\rangle \longrightarrow \langle H',\ \sigma',\ e'\rangle$.*

Proof. By case analysis on e:

- $e = v$: already a value (now includes constants c).
- $e = \mathbf{new}\ C()$: identical to λ_S^0.
- $e = \ell.m()$: by RT-Call, $\delta(s, m) = s'$ exists. R-Call applies (producing $c : msig(C, m)$ instead of **skip**, but applicability depends only on δ).
- $e = \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2$: standard split on whether e_1 is a value. □

6.4 Session Completion

Corollary 6.4 (Session Completion)**.** *If a well-typed λ_S^1 program $\langle \emptyset,\ \emptyset,\ e\rangle$ reduces to $\langle H,\ \sigma,\ v\rangle$, then $\sigma(\ell) = \bot$ for every $\ell \in \mathrm{dom}(H)$.*

Proof. By iterated preservation and the same argument as λ_S^0. The session-completion predicate examines only Γ' entries of the form $\ell : C\,@\,s$; base-type bindings never enter Γ', so the predicate is unaffected. □

7 Worked Examples

7.1 FileHandle with `read()` → `String`

Consider the FileHandle protocol from λ_S^0:

$$S_{\mathrm{FH}} = \&\{open : \&\{read : \&\{close : \mathbf{end}\},\ write : \&\{close : \mathbf{end}\}\}\}$$

In λ_S^1, we enrich the signature: $msig(\mathrm{FH}, read) = \mathbf{String}$. All other methods return **Unit**. The client program:

$$e_{\mathrm{read}} = \mathbf{let}\ f = \mathbf{new}\ \mathrm{FH}()\ \mathbf{in}\ \mathbf{let}\ _ = f.open()\ \mathbf{in}\ \mathbf{let}\ content = f.read()\ \mathbf{in}\ \mathbf{let}\ _ = f.close()\ \mathbf{in}\ content$$

Typing derivation.

$\Gamma_0 = \{f : \mathrm{FH}\,@\,s_0\}$	after T-New
$\Gamma_1 = \{f : \mathrm{FH}\,@\,s_1\}$	after $f.open() : \mathbf{Unit}$
$\Gamma_2 = \{f : \mathrm{FH}\,@\,s_2\}$	after $f.read() : \mathbf{String}$
$\Gamma_3 = \{f : \mathrm{FH}\,@\,s_4\}$	after $f.close() : \mathbf{Unit}$

The variable *content* receives the **String** value, but since **String** is a base type, *content* does *not* appear in any Γ_i. The final environment Γ_3 has f at $s_4 = \bot$: session completion holds.

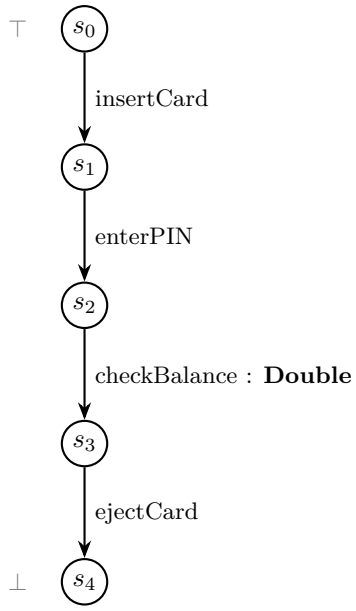

Figure 1: ATM state space. The return type annotation (Double) on the checkBalance transition is informational only—it does not affect the state-space structure or the lattice check.

Operational trace.

$$
\begin{aligned}
& \langle \emptyset,\ \emptyset,\ e_{\text{read}} \rangle \\
\longrightarrow\ & \langle H_0,\ \sigma_0[\ell \mapsto s_0],\ \textbf{let}\ _ = \ell.\mathit{open}()\ \textbf{in}\ \ldots \rangle \\
\longrightarrow\ & \langle H_0,\ \sigma_1[\ell \mapsto s_1],\ \textbf{let}\ \mathit{content} = \ell.\mathit{read}()\ \textbf{in}\ \ldots \rangle \\
\longrightarrow\ & \langle H_0,\ \sigma_2[\ell \mapsto s_2],\ \textbf{let}\ _ = \ell.\mathit{close}()\ \textbf{in}\ \text{“hello”} \rangle \qquad (c = \text{“hello”} : \textbf{String}) \\
\longrightarrow\ & \langle H_0,\ \sigma_3[\ell \mapsto s_4 = \bot],\ \text{“hello”} \rangle
\end{aligned}
$$

The return value “hello” flows through **let** *content* = ... **in** ... but the session store σ is advanced purely by δ.

7.2 ATM with `checkBalance()` → `Double`

Consider a simplified ATM (without selections, fitting the Branch-only restriction of $\lambda_S^{0,1}$):

$$S_{\text{ATM}} = \&\{\mathit{insertCard} : \&\{\mathit{enterPIN} : \&\{\mathit{checkBalance} : \&\{\mathit{ejectCard} : \textbf{end}\}\}\}\}$$

With $\mathit{msig}(\text{ATM}, \mathit{checkBalance}) = \textbf{Double}$ and all other methods returning **Unit**.

$$e_{\text{ATM}} = \textbf{let}\ a = \textbf{new}\ \text{ATM}()\ \textbf{in let}\ _ = a.\mathit{insertCard}()\ \textbf{in let}\ _ = a.\mathit{enterPIN}()\ \textbf{in let}\ b = a.\mathit{checkBalance}()\ \textbf{in}$$

The typing traces the states $s_0 \xrightarrow{\mathit{insertCard}} s_1 \xrightarrow{\mathit{enterPIN}} s_2 \xrightarrow{\mathit{checkBalance}} s_3 \xrightarrow{\mathit{ejectCard}} s_4 = \bot$. Variable b has type **Double** and is not session-tracked. Session completion holds.

8 Discussion and Conclusion

8.1 Implementation Correspondence

Table 1 extends the λ_S^0 correspondence table with the λ_S^1 additions.

The critical observation is the “Orthogonality” row: BICA Reborn’s `TypestateChecker` never inspects Java return types. It maintains a map from variables to integer state IDs and updates it purely via δ. λ_S^1 proves this is sound.

Table 1: λ_S^1 rule-to-implementation correspondence. Lines refer to `TypestateChecker.java` in BICA Reborn.

Rule	**Behaviour**	**BICA line**
T-Skip	Unit value, Γ unchanged	—
T-Const	Base-type constant, Γ unchanged	—
T-Var	Lookup $\Gamma(x)$	43
T-New	$\Gamma' = \Gamma, x : C \mathbin{@} \top$	99–101
T-Call	$\Gamma' = \Gamma[x \mapsto C \mathbin{@} s']$; returns $msig(C, m)$	52–65
T-Let	Thread Γ through e_1, e_2	42–66
Orthogonality	Return type ignored in state tracking	(entire checker)

8.2 Relation to VGR 2006

Vasconcelos, Gay, and Ravara [4] established the same orthogonality in their channel-based functional language: the typing judgement $\Gamma; \Sigma; e \mapsto \Sigma'; T; \Sigma''$ separates the result type T from the channel environment $\Sigma \to \Sigma', \Sigma''$. Their C-ReceiveD rule returns data type D while advancing the channel from $?D.S$ to S; the return type D flows through T, not through Σ.

λ_S^1 establishes the analogous result for the object setting: return types flow through T in $\mathcal{CT}; \Gamma \vdash e : T \dashv \Gamma'$, while session state changes flow through $\Gamma \to \Gamma'$.

8.3 Preview: λ_S^2 (Conditionals)

λ_S^1 allows the client to *bind* return values but not *inspect* them. Real ATM clients branch on the return value of `enterPIN()`: if `AUTH`, continue transacting; if `REJECTED`, eject card. This requires:

1. Conditionals: **if** x **then** e_1 **else** e_2.
2. Selection types: $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$.
3. The rule that selection labels are determined by return values.

These belong to λ_S^2, which will be a proper extension (not conservative) because it adds new expression forms and new session type constructors. λ_S^1 is the last calculus where session types and data types are fully orthogonal.

8.4 Mechanisation

λ_S^1 is mechanised in Lean 4 as an extension of the λ_S^0 formalisation (Step 200b). The key additions are:

- `BaseTy` inductive: **Int**, **Double**, **String**, **Bool**.
- Extended `Ty`: adds τ variant.
- `MethodSigTable`: $(C, m) \rightharpoonup \tau$.
- `tConst` and modified `tCall` constructors.
- `conservative_extension`: embedding λ_S^0 into λ_S^1.

The Lean formalisation confirms that all proofs from λ_S^0 extend to λ_S^1 with only the T-Const case and the modified T-Call return type as new proof obligations.

Acknowledgements. This work builds on the foundations established by Gay, Vasconcelos, and Ravara in their series of papers on session types for objects and channels [2, 4, 3].

References

[1] Alexandre Zua Caldeira. λ_s^0 — a formal calculus for straight-line typestate checking (step 200a), 2026. Research note.

[2] Simon Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert, and Alexandre Zua Caldeira. Modular session types for distributed object-oriented programming. In *POPL 2010*, pages 299–312. ACM, 2010.

[3] Simon J. Gay, Nils Gesbert, António Ravara, and Vasco T. Vasconcelos. Modular session types for objects. *Logical Methods in Computer Science*, 11(4:12):1–76, 2015.

[4] Vasco T. Vasconcelos, Simon J. Gay, and António Ravara. Type checking a multithreaded functional language with session types. *Theoretical Computer Science*, 368(1–2):64–87, 2006.